A Comparative Study of Shared-Memory, GPU, and Distributed Parallelisation Strategies for 2D Image Convolution

Project Report — Parallel Computing Architecture and Programming (PCAP)

Saatvik Das, Saket Rama, Zaid Khan
Department of Computer Science and Engineering
Manipal Institute of Technology, Bengaluru

Guide: Prof. Anil Kumar
Semester V

***Abstract*—**Two-dimensional convolution is the computational core of classical image processing and of the convolutional layers of modern computer vision, and a textbook memory-bound stencil. This project implements one algorithm, 2D convolution with an odd-sized K×K filter, as a serial C++ baseline, an OpenMP version, a naive CUDA kernel, a CUDA kernel with the filter in constant memory, a CUDA kernel that adds shared-memory tiling with halo regions, and an MPI strip decomposition. Every output is validated against the serial reference (largest error 4.8×10^{-7} against a 10^{-4} limit across 728 benchmark runs on 4K and 8K images, K = 3 to 15). On an Intel i7-12650H and an RTX 3050 Laptop GPU, OpenMP reaches 6.7× to 8.5× over serial at 16 threads and stops scaling near 4 threads. The tiled kernel is 0.9× to 1.9× faster than the naive kernel, the gain growing with K. Nsight Compute explains why: DRAM traffic is identical across the three kernels because the caches already capture the reuse, while tiling cuts global-load traffic through L1 by up to 86×. Host–device transfers take 70% to 93% of the end-to-end GPU time, so a kernel-only speedup of up to 389× becomes 104× for the whole job. Measured serial fractions bound end-to-end OpenMP speedup to 1.1×–2.6×, and the observed values sit within 2% to 16% of that bound. The deliverable is a reproducible benchmark suite, analysis scripts, a local demonstration page and this report.

***Index Terms*—**2D convolution, OpenMP, CUDA, shared memory, constant memory, MPI, Amdahl's law, roofline model, Nsight Compute, GPU acceleration, memory bandwidth, parallel computing.

I. Introduction

For roughly four decades application performance improved automatically as transistor density and clock frequency rose together. That ended in the mid-2000s, when Dennard scaling broke down and clock frequencies settled near 3–5 GHz. Vendors responded by scaling outward: multi-core CPUs, wide SIMD units, many-core GPUs and clusters. Performance is therefore no longer inherited; it must be engineered, and one algorithm can differ by two or three orders of magnitude in runtime depending only on how it is mapped to hardware.

Image convolution is an unusually good subject for studying this. It is embarrassingly parallel at the output level, because every output pixel is independent, so it exposes the programming models without algorithmic distractions. It is not trivially fast, however: its sliding-window access pattern overlaps heavily between neighbouring outputs. That overlap is both the problem (redundant memory traffic) and the solution (reuse through blocking and tiling). The scarce resource on contemporary hardware is memory bandwidth, not arithmetic throughput.

This report presents ConvoX, a benchmark suite that runs one convolution algorithm on five execution paths under identical input, validation and timing conditions, and attributes every performance change to a specific hardware mechanism. It reports the full measurement campaign (728 validated runs), a profiler-based explanation of the GPU results, and an honest accounting of data-transfer cost.

II. Problem Statement

High-performance convolution implementations are usually delivered as tuned black boxes (cuDNN, OpenCV, oneDNN, FFT- and Winograd-based methods): the user calls a function and gets a number, with no visibility into which optimisation produced the speedup. Published comparisons are typically confined to one programming model, use small images and 3×3 filters that fit in cache, and report only kernel time, omitting host–device transfer and so overstating the realistic GPU advantage. Amdahl's law predictions are rarely checked against a measured serial fraction on the same workload. No compact, reproducible study implements one identical algorithm across serial, shared-memory, naive-GPU, optimised-GPU and distributed-memory models, sweeps both filter size and launch configuration, and attributes the differences to levels of the memory hierarchy. ConvoX provides that study.

III. Literature Survey

Amdahl [1] bounded the speedup of a program by its non-parallelisable fraction, $S(N) = 1/(f + (1-f)/N)$, the standard first-order scaling model. It assumes a fixed problem size and ignores parallel overheads and bandwidth saturation; Gustafson [2] argues the problem size should grow with the machine instead. Williams et al. [3] gave the roofline model, which bounds attainable performance by the lesser of peak arithmetic rate and memory bandwidth times arithmetic intensity.

Datta et al. [6] auto-tuned stencil kernels on multicore CPUs and early GPUs and showed they are bandwidth-bound, so cache and register blocking rather than extra cores drive most of the gain; their 3D finite-difference targets predate unified L2 caches and they did not isolate constant memory for coefficients. Micikevicius [7] introduced the shared-memory tiling pattern with explicit halo loading and `__syncthreads()` phases for 3D stencils, but compared no CPU or OpenMP baseline. Volkov and Demmel [8] showed that maximum occupancy is not

maximum performance, which motivates the block-dimension sweep here.

cuDNN [9] reformulates convolution as im2col plus GEMM, which is fast but expands the intermediate matrix and is tuned for small filters and batched tensors, not large single-image kernels up to 15×15. Lavin and Gray [10] apply Winograd minimal filtering to cut multiplications for small filters; the benefit fades as K grows and hides memory-hierarchy behaviour behind arithmetic restructuring. None of these works compares serial, OpenMP and several CUDA stages on one algorithm with transfer cost and profiler evidence, which is the gap ConvoX fills.

IV. Aim and Objectives

A. Aim

To design, implement and empirically evaluate progressively parallelised implementations of 2D image convolution, namely serial, OpenMP, naive CUDA and memory-hierarchy-optimised CUDA (with an optional MPI layer), in order to quantify the contribution of each parallelisation and memory-optimisation technique and to check the observed scaling against established theoretical models.

B. Objectives and outcome

TABLE I
Objectives and where each was met

#	Objective	Result
1	Serial reference, K = 3–15, zero-pad / clamp border	Done; oracle for every check (Sec. V)
2	OpenMP rows, collapse(2), schedules, 1–16 threads	Done (Sec. VII-A, B)
3	Naive CUDA and block-dimension study	Done (Sec. VII-C)
4	Constant memory, shared tiling, profiler evidence	Done; DRAM traffic measured (Sec. VII-D)
5	4K and 8K sweep, kernel-only vs end-to-end, Amdahl	Done; 728 runs (Sec. VII-F, G)
6	Optional MPI strip decomposition	Done; verified for 1, 2, 3, 4, 7 ranks (Sec. VII-H)

V. System Architecture and Methodology

A. Algorithm

For an input image I of H×W pixels and a filter F of size K×K (K odd, radius R = (K−1)/2), the output is

$$O[y][x] = \Sigma_{i=-R}^{R} \Sigma_{j=-R}^{R} I[y+i][x+j] \cdot F[i+R][j+R] \quad (1)$$

Pixels outside the image are zero (clamp-to-edge is a compile-time alternative). All arithmetic is single-precision `float`. The filter is a normalised Gaussian by default; Sobel and box filters are provided for visual verification. Image libraries (stb_image) decode and encode files only and never perform the convolution.

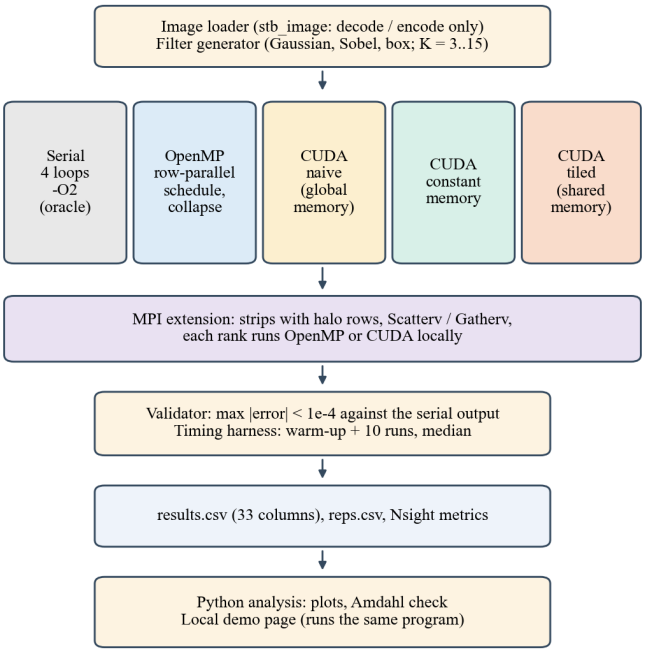


Fig. 1. ConvoX pipeline. All execution paths share one loader, filter generator, validator and timing harness.

B. Implementation stages

Serial. Four nested loops (output row, output column, filter row, filter column), compiled at `-O2` with auto-vectorisation disabled, so the compiler does not parallelise the reference. It defines T_{serial}, the denominator of every speedup, and the reference output.

OpenMP. `parallel for` over output rows with serial filter loops inside; variants use `collapse(2)`, schedule static, dynamic with chunk 1, 8, 32, and guided, and thread counts 1, 2, 4, 6, 8, 12 and 16 (up to the machine's 16 hardware threads).

CUDA naive. One thread per output pixel; the grid is $\text{ceil}(W/B_X) \times \text{ceil}(H/B_Y)$ with bounds guards. Image and filter are both read from global memory.

CUDA constant. The filter moves to `__constant__` memory so that all threads of a warp read the same coefficient through the broadcast path.

CUDA tiled. Each block loads a $(T+K-1)^2$ input tile with halo into shared memory, calls `__syncthreads()`, then computes from shared memory. The three kernels are kept separate so each optimisation is attributable on its own.

MPI. Rank 0 scatters horizontal strips of H/N + 2R rows (the 2R rows are the halo) with `MPI_Scatterv`; each rank runs a local OpenMP or CUDA kernel; interior rows return through `MPI_Gatherv`. It is intentionally simple, not communication-optimal.

C. Experimental design

Table II lists every parameter that was varied; the sweep is the full cross product of the configurations below.

TABLE II
Parameters swept

Parameter	Values
Images	4 synthetic: 2 at 3840×2160, 2 at 7680×4320
Filter size K	3, 5, 7, 9, 11, 13, 15 (Gaussian)
OpenMP threads	1, 2, 4, 6, 8, 12, 16
OpenMP schedule	static, dynamic(1/8/32), guided; rows or collapse(2)
CUDA block	8×8, 16×16, 32×32
CUDA variant	naive, +constant, +constant +shared tiles
Total	26 configurations × 4 images × 7 K = 728 runs

Convolution cost does not depend on pixel values, so procedural images (smooth regions plus sharp edges, generated by a seeded script) give exact resolutions with no licensing questions and make blur and edge outputs easy to judge by eye.

D. Timing and validation

CPU timing uses `std::chrono::steady_clock`. GPU kernels are timed with CUDA events; host-to-device copy, kernel and device-to-host copy are recorded separately, with a device synchronise before any host timer stops. Every configuration runs one discarded warm-up and ten timed iterations and reports the median, which resists scheduling noise and clock-frequency transients. Each output is compared element-wise with the serial output; because summation order differs, exact equality is not expected, and the criterion is a maximum absolute error below 10^{-4} with the RMS error recorded alongside.

E. Analytical framework

The naive kernel performs $2K^2$ floating-point operations per output pixel while loading $4K^2$ bytes, an arithmetic intensity of about 0.5 FLOP/byte for every K. The tiled kernel loads $(T+K-1)^2/T^2$ elements per output, so for T = 16 and K = 5 the load count falls from 25 to about 1.56, a reuse factor near 16. The serial fraction f is measured, not assumed: the parts that never parallelise (image decode, filter build, output allocation and PNG encode) are timed separately and compared with the serial convolution time c, $f = s/(s + c)$.

F. Environment

TABLE III
Measurement platform

Item	Value
CPU	Intel Core i7-12650H, 6 P + 4 E cores, 16 threads
GPU	NVIDIA RTX 3050 Laptop, 4 GB, cc 8.6; peak about 188 GB/s, 6.1 TFLOP/s
Software	WSL2 Ubuntu 24.04, GCC 13.3 (-O2), CUDA 12.6, OpenMPI 4.1.6, CMake 3.28, Python 3.12
Profiling	Nsight Compute (DRAM, L1, L2, occupancy)

VI. Implementation and Testing

The code base is organised so that each part can be read and presented on its own: `src/common` (image, filter, validator, timing, CSV, runner, sweep driver and command line), `src/serial`, `src/openmp`, `src/cuda`, `src/mpi`, `analysis` (plots, Amdahl, Nsight driver), `tests`, `scripts` and `ui`. It comprises about 1,700 lines of C++/CUDA, 500 lines of tests, 540 lines of analysis code and 1,300 lines for the demonstration page. A single `convox` program offers `info`, `run`, `sweep` (resumable) and `profile` commands.

Eight test programs cover image I/O, filter generation, the validator, the serial reference, the timing harness, the sweep driver, all 432 OpenMP variants and all 360 CUDA variants, including odd image sizes and invalid launch configurations. The MPI program was checked against serial for 1, 2, 3, 4 and 7 ranks with all three local kernels at K = 3 and 15, including row counts that do not divide evenly. In the final sweep all 728 runs passed validation with a largest absolute error of 4.8×10^{-7}.

A local demonstration page runs the same `convox` binary on any uploaded image, shows the original and filtered pictures, a results table and charts, and offers optional thread, block-shape, filter-size and MPI experiments. It never fabricates numbers: each value is the median reported by the benchmark program, validated against serial.

VII. Results and Discussion

Unless stated, figures are for the 8K image (7680×4320), a Gaussian filter and the median of 10 runs. Speedup is serial time divided by the version's time on the same image and K.

A. OpenMP scaling

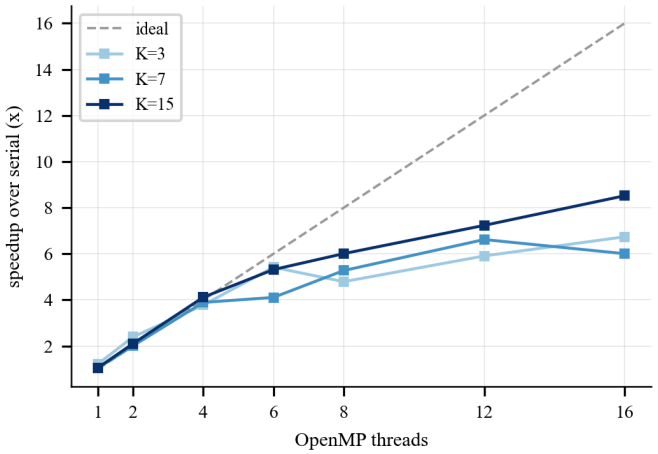


Fig. 2. OpenMP speedup over serial against thread count (static schedule, row parallelism).

Scaling is close to linear to 4 threads (about 3.8×–4.1×), then flattens: 6.7× at K=3, 6.0× at K=7 and 8.5× at K=15 with 16 threads. Efficiency at 16 threads is 0.38 to 0.53, lower for small filters, which do little arithmetic per byte and so contend for memory bandwidth sooner. The CPU is hybrid, so threads beyond 6 land on efficiency cores or on hyper-threads of busy cores; this machine cannot separate bandwidth saturation from that effect.

B. Schedules and collapse

TABLE IV
OpenMP schedules, 8 threads, K = 7 (ms)

Schedule	rows	collapse(2)
static	271	288
dynamic, chunk 1	238	1209
dynamic, chunk 8	244	324
dynamic, chunk 32	247	266
guided	246	255

The synopsis expected static to win on uniform rows. Here dynamic and guided are 9%–12% faster, plausibly because a hybrid CPU has unequal cores so equal static shares finish unevenly; this was not isolated. The clear result is the cost of

fine-grained distribution: collapse(2) with chunk 1 hands out single pixels and is five times slower.

C. CUDA block dimensions

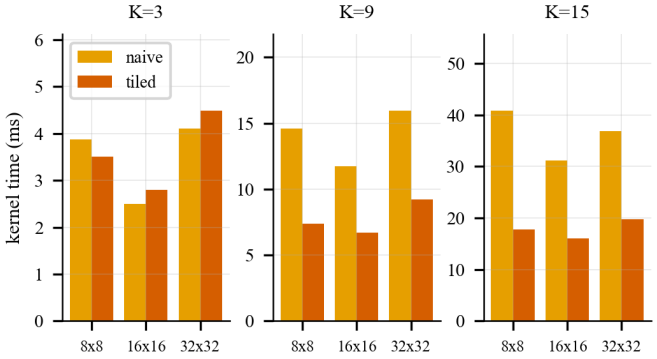


Fig. 3. CUDA kernel time against block shape for K = 3, 9 and 15.

16×16 is fastest for every kernel and K shown. An 8×8 block holds only two warps and, for the tiled kernel, has a large halo relative to its tile; 32×32 uses the 1024-thread maximum and a larger shared tile, which limits resident blocks per multiprocessor. Nsight reports achieved warp occupancy of 71%–88% for the naive kernel and 94%–97% for the tiled kernel at 16×16.

D. Constant memory and tiling

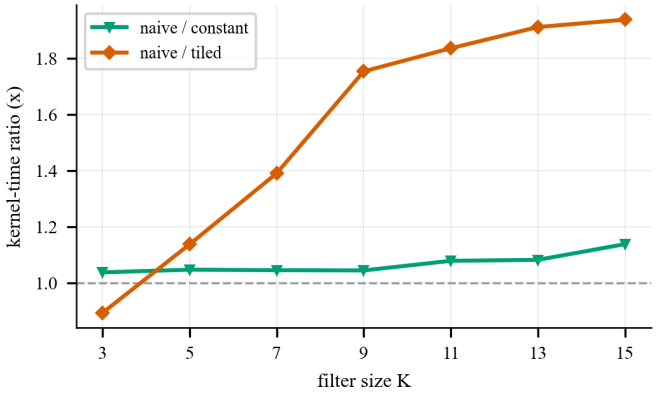


Fig. 4. Kernel-time ratio of naive to optimised kernels at 16×16 blocks.

The naive/tiled ratio is 0.89 at K=3 (tiling is slightly slower), 1.39 at K=7 and 1.94 at K=15: the redundancy removed grows as K^2 while the halo overhead grows with the tile perimeter, so the gain appears only for larger filters. Constant memory alone helps by 3%–12% in the sampled cases.

TABLE V
Nsight Compute, 16×16, 8K image

K	Kernel	DRAM rd MB	DRAM wr MB	L1 loads MB	Warps %	ms
3	naive	132.8	132.1	1889	70.9	3.2
3	tiled	132.9	132.1	322	93.6	3.6
7	naive	133.1	132.2	10905	81.9	11.3
7	tiled	133.6	132.1	408	96.3	8.2
15	naive	137.9	132.5	51192	88.3	47.7
15	tiled	147.6	132.3	597	97.4	25.0

This corrects an expectation in the synopsis. The 8K image is 133 MB and every kernel reads about that much and writes the same amount: each pixel crosses DRAM once, because the L1 and L2 caches already serve the K^2 reuse of the naive kernel. Tiling does not reduce DRAM traffic. It removes traffic between cache and multiprocessor, 51 GB down to 0.6 GB at K=15 (86×), and the speedup comes from fewer load instructions and higher occupancy. Profiler kernel times exceed harness times because the profiler serialises launches and fixes clocks.

E. Roofline

Analytical intensity is about 0.5 FLOP/byte for the naive kernel and $2K^2/(4(T+K-1)^2/T^2)$ for the tiled kernel, 3.6 to 32 FLOP/byte at T = 16. Naive points plot above the memory roof because the real DRAM intensity is much higher than the analytical one, for the cache reason above. Tiled points move right and, at K=15, reach about 930 GFLOP/s, 15% of the 6.1 TFLOP/s peak. They stay well below both roofs, so load and shared-memory instruction throughput are the likely limiters; this was not measured.

F. Compute time, transfers and end-to-end speedup

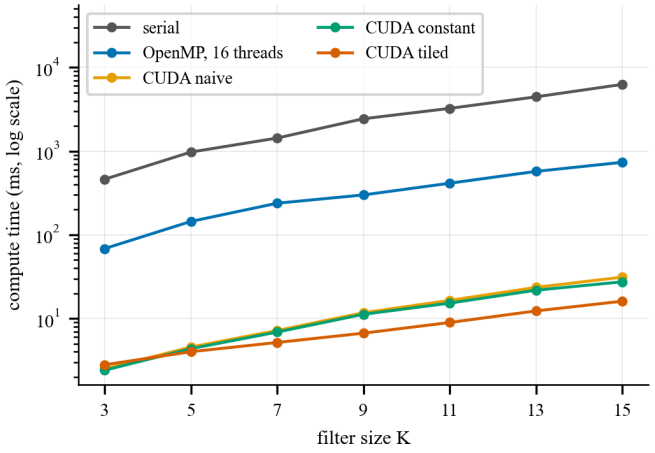


Fig. 5. Compute-only time against filter size for every version (8K image, best configuration).

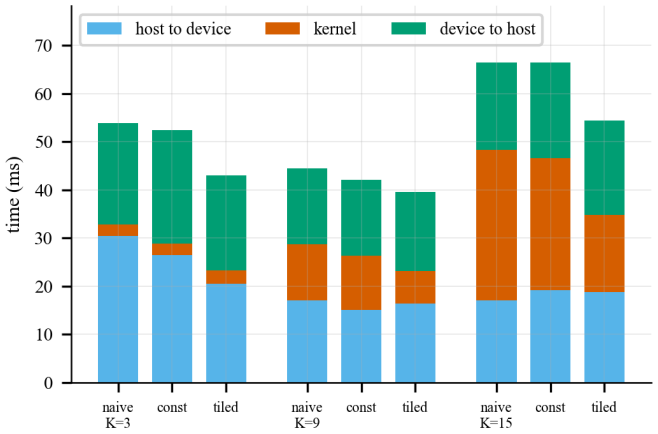


Fig. 6. Where GPU time goes: host-to-device copy, kernel and device-to-host copy (16×16 blocks).

Host–device copies are 93% of the end-to-end time of the tiled kernel at K=3 and 70% at K=15. Reporting kernel time alone therefore overstates the practical gain.

TABLE VI
GPU speedup over serial (x)

K	Naive kernel	Naive end-to-end	Tiled kernel	Tiled end-to-end
3	184	7.8	165	9.1
7	199	35.0	277	36.9
15	201	86.3	389	103.7

Even end-to-end the GPU beats 16 OpenMP threads at every K (9.1× against 6.7× at K=3, 104× against 8.5× at K=15), but the margin is small for small filters. Offload pays when arithmetic per transferred byte is high: large filters, batches, or pipelines that keep data on the device. Transfers use pageable memory (plain `cudaMemcpy`); pinned memory would shorten them.

G. Amdahl's law

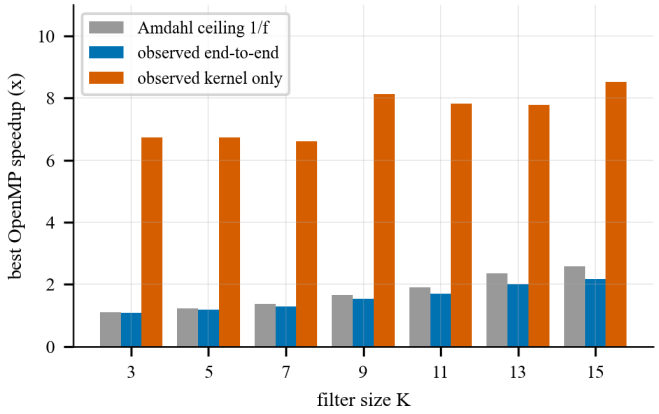


Fig. 7. Amdahl ceiling predicted from the measured serial fraction against the best observed OpenMP speedups.

The serial fraction is dominated by image decode (about 0.8–1.4 s for 8K) and PNG encode (2.5–3.2 s), giving f = 0.90 at K=3 and 0.39 at K=15, ceilings of 1.11× to 2.58×, and observed end-to-end speedups 2%–16% below the ceiling. The kernel-only speedup, which the law with f = 0 would put at 16× for 16 threads, reaches only 6.6×–8.5×. The law does not model that gap (bandwidth contention, hybrid cores, synchronisation) nor PCIe transfer for the GPU.

H. MPI extension

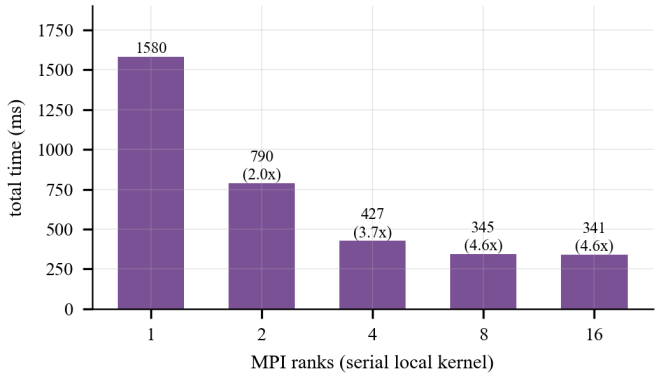


Fig. 8. MPI strip decomposition, serial local kernel, 8K image, K=7, one node (total time with scatter and gather).

Pure MPI scales almost linearly to 4 ranks, then flattens near 5×: 1550 ms of compute at 1 rank falls to 306 ms at 16. Scatter and gather add 25–35 ms. Splitting OpenMP threads across ranks gives no gain, since the threads already saturated memory, and with CUDA all ranks share one GPU and serialise on it. These runs demonstrate decomposition and halo handling, not network scaling: no cluster was available.

I. Visual check

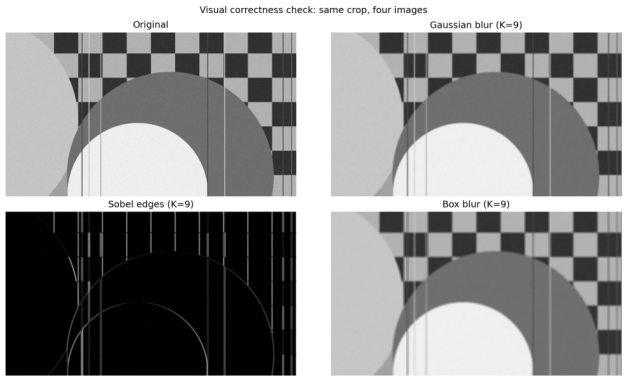


Fig. 9. Same crop of the original, the Gaussian-blurred and the Sobel edge outputs.

The outputs look as expected, which complements the numerical validation.

VIII. Key Findings

1) Memory behaviour, not parallelism alone, sets performance: the CPU stops scaling after 4 threads and the tiled kernel reaches 15% of arithmetic peak.
2) The caches matter more than the synopsis anticipated: naive CUDA already needs about one DRAM pass, so tiling pays through cache-to-multiprocessor traffic and occupancy, and only once K is large.
3) Transfers change the conclusion: a kernel-only 389× becomes 104× end-to-end, and about 8× at small K.
4) Amdahl's law is a useful first-order bound, here dominated by image I/O; its failure to predict the kernel-only plateau is the instructive part.

IX. Limitations

- One machine: a hybrid CPU, a 4 GB laptop GPU that can throttle, and WSL2 on Windows. Results do not transfer directly to other hardware.
- Threads above 6 mix efficiency cores and hyper-threads, so the cause of the OpenMP plateau is not isolated.
- The schedule and collapse sweep ran at 8 threads only; static being slower than dynamic is a measurement on this CPU, not a rule.
- Images are synthetic and grayscale; reported runs use zero-padding (clamp-to-edge builds and tests exist).
- A GPU driver reset interrupted the sweep once and one run hung once; the sweep was resumed from the recorded runs.
- MPI was measured on a single node, so it carries no network cost.
- Profiler data cover one 8K image at K = 3, 7, 15 with 16×16 blocks.

X. Conclusion and Future Work

ConvoX delivers a validated, reproducible comparison of one convolution algorithm across serial, OpenMP, three CUDA stages and MPI. The measurements confirm that convolution is memory-bound on the CPU, show that tiling's advantage on a modern GPU comes from cache-to-multiprocessor traffic and occupancy rather than DRAM savings, and show that transfer cost reduces a 389× kernel speedup to about 104× for a single image. Future work includes pinned memory and CUDA streams to overlap transfers, batched and colour images, a vendor-library baseline, separating the hybrid-core effect with core pinning, and measurements on a multi-node cluster.

Reproduction. `bash scripts/build.sh`, `bash scripts/run_tests.sh`, `convox sweep`, then the summary, plot and Amdahl scripts. The source is at github.com/PCAP-ZSS/ConvoX.

References

[1] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, Atlantic City, NJ, USA, 1967, pp. 483–485.

[2] J. L. Gustafson, "Reevaluating Amdahl's law," *Commun. ACM*, vol. 31, no. 5, pp. 532–533, May 1988.

[3] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[4] D. B. Kirk and W.-m. W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, 4th ed. Cambridge, MA, USA: Morgan Kaufmann, 2022.

[5] L. Dagum and R. Menon, "OpenMP: An industry standard API for shared-memory programming," *IEEE Comput. Sci. Eng.*, vol. 5, no. 1, pp. 46–55, Jan.–Mar. 1998.

[6] K. Datta *et al.*, "Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures," in *Proc. ACM/IEEE Conf. Supercomputing (SC '08)*, Austin, TX, USA, 2008, pp. 1–12.

[7] P. Micikevicius, "3D finite difference computation on GPUs using CUDA," in *Proc. 2nd Workshop on General Purpose Processing on Graphics Processing Units (GPGPU-2)*, Washington, DC, USA, 2009, pp. 79–84.

[8] V. Volkov and J. W. Demmel, "Benchmarking GPUs to tune dense linear algebra," in *Proc. ACM/IEEE Conf. Supercomputing (SC '08)*, Austin, TX, USA, 2008, pp. 1–11.

[9] S. Chetlur *et al.*, "cuDNN: Efficient primitives for deep learning," *arXiv preprint arXiv:1410.0759*, 2014.

[10] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 4013–4021.

[11] NVIDIA Corporation, *CUDA C++ Programming Guide*, Version 12.x. Santa Clara, CA, USA: NVIDIA, 2024.

[12] NVIDIA Corporation, *CUDA C++ Best Practices Guide*, Version 12.x. Santa Clara, CA, USA: NVIDIA, 2024.

[13] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, Version 5.2, Nov. 2021.

[14] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard*, Version 4.0, Jun. 2021.

[15] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.

[16] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.